

Movie Ticketing System

Software Requirements Specification

Version 0.3

09/24/2026

Group 11

Cameron Penaranda, Justin Paul Liporada,
Ayaan Bhatkar

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
9/22/2026	Version 0.1	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	First revision,
9/24/2026	Version 0.2	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Second revision, completed functionalities and use cases.
	Version 0.3	Cameron Penaranda, Justin Paul Liporada, Ayaan Bhatkar	Added Software Design Specification

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY.....	II
DOCUMENT APPROVAL.....	II
1. INTRODUCTION.....	1
1.1 PURPOSE.....	1
1.2 SCOPE.....	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	1
1.4 REFERENCES.....	1
1.5 OVERVIEW.....	2
2. GENERAL DESCRIPTION	3
2.1 PRODUCT PERSPECTIVE.....	3
2.2 PRODUCT FUNCTIONS.....	3
2.3 USER CHARACTERISTICS.....	3
2.4 GENERAL CONSTRAINTS	2
2.5 ASSUMPTIONS AND DEPENDENCIES	4
3. SPECIFIC REQUIREMENTS	5
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	5
3.1.1 <i>User Interfaces</i>	5
3.1.2 <i>Hardware Interfaces</i>	5
3.1.3 <i>Software Interfaces</i>	5
3.1.4 <i>Communications Interfaces</i>	5
3.2 FUNCTIONAL REQUIREMENTS	5
3.2.1 <i><Functional Requirement or Feature #1></i>	5
3.2.2 <i><Functional Requirement or Feature #2></i>	6
3.3 USE CASES	6
3.3.1 <i>Use Case #1</i>	4
3.3.2 <i>Use Case #2</i>	10
3.4 CLASSES / OBJECTS	11
3.4.1 <i><Class / Object #1></i>	11
3.4.2 <i><Class / Object #2></i>	11
3.5 NON-FUNCTIONAL REQUIREMENTS	11
3.5.1 <i>Performance</i>	11
3.5.2 <i>Reliability</i>	12
3.5.3 <i>Availability</i>	12
3.5.4 <i>Security</i>	12
3.5.5 <i>Maintainability</i>	12
3.5.6 <i>Portability</i>	12
3.6 INVERSE REQUIREMENTS	12
3.7 DESIGN CONSTRAINTS	12
3.8 LOGICAL DATABASE REQUIREMENTS	13
3.9 OTHER REQUIREMENTS.....	13
4. ANALYSIS MODELS	14
4.1 SEQUENCE DIAGRAMS.....	14
4.3 DATA FLOW DIAGRAMS (DFD)	14
4.2 STATE-TRANSITION DIAGRAMS (STD)	14
5. CHANGE MANAGEMENT PROCESS.....	14
A. APPENDICES	14

Movie Ticketing System

A.1 APPENDIX 1	14
A.2 APPENDIX 2	14

1. Introduction

1.1 Purpose

The purpose of this SRS document is to define the functional and non-functional requirements of the Movie ticketing system. The document establishes what the system is expected to do, how users will interact with it, what external interfaces may be required, and what constraints must be considered during development. This SRS is intended to provide an understanding of the Movie ticketing system for the development team, testers, project stakeholders, and the CS250 instructor. Developers can use the requirements to help guide them through the implantations, while testers can use them to verify that the completed system satisfies the specified requirements.

1.2 Scope

The Movie Ticketing System is a software system that allows customers to create an account, browse available movies from all movie theaters, available showtimes and auditoriums for currently airing movies, seat selections, snacks for the movie, and purchasing of everything through credit, debit, and PayPal. The user's receipt of sale is then shown to them and has the option to send a copy of their receipt to their email address with possible refunds. The Movie Ticketing System will not accept EBT.

1.3 Definitions, Acronyms, and Abbreviations

SRS	Software Requirement Specifications
EBT	Electronic Benefits Transfer
CS250	Introduction to Software Systems
API	Application Programming Interface

1.4 References

This subsection should:

- (1) Provide a complete list of all documents referenced elsewhere in the SRS, or in a separate, specified document.*
- (2) Identify each document by title, report number - if applicable - date, and publishing organization.*
- (3) Specify the sources from which the references can be obtained.*

This information may be provided by reference to an appendix or to another document.

<text>

1.5 Overview

The remainder of this SRS describes the Movie Ticketing System's general characteristics, functional requirements, external interfaces, use cases, classes and objects, non-function requirements, database requirements, analysis models, and change management process.

2. General Description

2.1 Product Perspective

The Movie Ticketing System is intended to provide a centralized application that customers can locate nearby participating movie theaters, browse available movies and showtimes, select seats, purchase movie tickets, and purchase available food and drink items. The system will maintain information concerning theaters, theater locations, movies, showtimes, seating arrangements, concession items, and customer purchases. Location services may be used to determine the customer's current location and identify nearby theaters. If location access is unavailable or denied, customers will be able to search for theaters manually. The system may also interact with an external payment processing service when completing customer purchases. Administrators will have access to separate functionality for maintaining movie, showtime, and concession information.

2.2 Product Functions

The Movie Ticketing System will provide customers with the ability to search for movies. After selecting a movie, customers will be able to view movie information and the available showtimes at various theaters and be able to select a showtime for ticket purchase. The system will display the seating arrangement for the selected showtime and allow customers to select available seats. Customers will also be able to browse food and drink items offered by the selected theater and add available concession items to their order. Before completing a purchase, the system will calculate and display the total cost of the selected tickets and concession items. After a successful transaction, the system will generate purchase confirmation information. Authorized administrators will be able to manage movie information, showtimes, and concession offerings. The template describes this subsection as a summary of the major functions performed by the software.

2.3 User Characteristics

The primary users of the Movie Ticketing System are customers and administrators. Customers are expected to have basic familiarity with using a computer, smartphone, or similar electronic device, but they will not require specialized technical knowledge to use the system. Customers will use the application to search for theaters, browse movies and showtimes, select seats, purchase tickets, browse food and drink items, and complete orders. Administrative users are expected to have basic knowledge of theater operations and the ability to operate a graphical software application. Administrators will be responsible for maintaining information such as available movies, scheduled showtimes, and concession offerings. Administrative functionality will be restricted to authorized users and will not be accessible to ordinary customers.

2.4 General Constraints

The Movie Ticketing System must be developed within the schedule and requirements established for the CS 250 project and must provide a graphical user interface through which customers and administrators can interact with the system. The system must maintain persistent

information concerning participating theaters, movies, showtimes, auditoriums, seats, concession items, and completed purchases. Location-based theater discovery will depend on the customer's device supporting location services and the customer granting permission to access that information. Because location permission may be denied or unavailable, the system must also support manual theater searching. Each showtime must correspond to a specific movie, theater, auditorium, date, and time, and the same seat must not be sold to multiple customers for the same showtime. Movie, showtime, and food and drink availability may differ between participating theaters, and administrative functions must only be accessible to authorized administrators. The template identifies this section as the place to describe factors that limit the developers' options when designing the system.

2.5 Assumptions and Dependencies

The Movie Ticketing System assumes that customers have access to a compatible computing device and the network connection necessary to use the application. Participating theaters are assumed to provide accurate information regarding their locations, available movies, scheduled showtimes, auditorium seating arrangements, and available food and drink items. Each theater is assumed to contain one or more defined auditoriums, and each auditorium is assumed to contain a predefined collection of seats. Each showtime is assumed to be associated with a valid movie, theater, and auditorium. Location-based functionality depends on location services supplied by the customer's device, browser, or operating system, although customers may manually provide a location if these services are unavailable or permission is denied. The system also depends on persistent data storage for maintaining theater, movie, showtime, seat, concession, and purchase information. If an external payment processing service is used, successful online purchasing will additionally depend on the availability of that service. These assumptions represent external conditions that may cause the system requirements to change if they are no longer valid.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Movie Ticketing System shall provide a graphical user interface in which customers can browse movies, view showtimes, select seats, purchase tickets, and view confirmations.

The customer interface will include:

- Theater discovery
- Theater search
- Movie browsing
- Movie information
- Showtime selection
- Seat selection
- Food and drink browsing
- Order summary
- Checkout
- Purchase confirmation

The location interface shall request permission before the user's device location. The system will provide a manual theater-search interface when location service is unavailable or denied.

3.1.2 Hardware Interfaces

The Movie Ticketing System does not require any specialized hardware interfaces. It will operate using standard computing hardware that is capable of running the standalone application.

The system may indirectly interact with device hardware that is used by location services, such as GPS or other positioning technologies through the operating system or browser-provided location functionality.

3.1.3 Software Interfaces

The Movie Ticketing System shall interface with a persistent database for storing and retrieving movie, theater, showtime, seat, customer, and ticket information. The system may also interface with an external payment processing service for payment authorization.

3.1.4 Communications Interfaces

The Movie Ticketing System shall communicate between the client application and the server using network-based communication.

Communication involving sensitive information shall use secure network communication.

3.2 Functional Requirements

3.2.1 Browse movies

3.2.1.1 Introduction

The system shall allow customers to browse movies currently available for ticket purchase.

3.2.1.2 Inputs

Movie Ticketing System

The system shall accept a customer's request to view available movies.

3.2.1.3 Processing

The system shall retrieve movies currently available from persistent storage.

3.2.1.4 Outputs

The system shall display each available movie's title and associated movie information.

3.2.1.5 Error Handling

If no movies are available, the system shall notify the customer that no movies are currently available.

3.2.2 Locate Nearby Theaters

3.2.2.1 Introduction

The system shall allow customers to locate participating movie theaters near their current location.

3.2.2.2 Inputs

The system shall allow the user to request nearby theaters.

The system shall request permission before accessing the user's current location.

The system shall allow the user to manually enter a search location.

3.2.2.3 Processing

When location permission is granted, the system shall obtain the user's current location through supported location services.

The system shall compare the search location with participating theater locations.

3.2.2.4 Outputs

The system shall display participating theaters located near the user's requested location.

The system shall allow the customer to select a displayed theater.

3.2.2.5 Error Handling

If location permission is denied, the system shall provide manual theater search.

If no participating movie theaters are found for the requested location, the system shall notify the customer.

Failure of location services shall not prevent the user from manually searching for theaters.

3.2.3 View showtimes

3.2.3.1 Introduction

The system shall allow customers to view available showtimes for a selected movie.

3.2.3.2 Inputs

The customer shall select a movie.

Movie Ticketing System

3.2.3.3 Processing

The system shall retrieve all valid showtimes associated with the selected movie.

3.2.3.4 Outputs

The system shall display the date and start time associated with the selected movie.

3.2.3.5 Error Handling

If the selected movie has no scheduled showtimes, the system shall notify the customer that there are no available showtimes available.

3.2.4 Select Seats

3.2.4.1 Introduction

The system shall allow customers to select seats for a selected movie showtime.

3.2.4.2 Inputs

The system shall accept the customer's selection of one or more available seats.

3.2.4.3 Processing

The system shall retrieve the seating arrangement for the selected showtime and determine which seats are currently available.

3.2.4.4 Outputs

The system shall retrieve the seating arrangement and distinguish available seats from unavailable seats.

3.2.4.5 Error Handling

If a customer attempts to select an available seat, the system shall notify the customer that the seat is unavailable and require another seat to be selected.

3.2.5 Browse Food and Drink

3.2.5.1 Introduction

The system shall allow customers to browse food and drink items available at the selected theater.

3.2.5.2 Inputs

The customer shall select a theater before browsing theater-specific concession items.

3.2.5.3 Processing

The system shall retrieve food and drink items available at the selected theater.

3.2.5.4 Outputs

The system shall display food and drink items.

The system shall display the price of concession items.

The system shall allow the user to add concession items to the current order.

The system shall allow the user to add concession items to the current order.

The system shall allow the user to change the quantity of concession items in their current order.

3.2.5.5 Error Handling

The system shall prevent customers from ordering concession items that are unavailable.

If there are no concession items available at the participating theater, the system shall notify the user.

3.2.6 Purchase Tickets

3.2.6.1 Introduction

The system shall allow customers to purchase tickets for a selected movie showtime.

3.2.6.2 Inputs

The system shall accept the customer's selected showtime, selected seats, and required payment information.

3.2.6.3 Processing

The system shall verify that the selected seats are available, calculate the total ticket price, and process the customer's payment.

3.2.6.4 Outputs

The system shall display the total purchase price and whether the ticket purchase was successfully completed.

3.2.6.5 Error Handling

If payment processing fails for a selected seat or is no longer available, the system shall notify the customer and shall not complete the purchase.

3.2.7 Purchase Confirmation

3.2.7.1 Introduction

The system shall provide the customer with confirmation after a successful ticket purchase.

3.2.7.2 Inputs

The system shall receive the information associated with the successfully completed purchase.

3.2.7.3 Processing

The system shall generate a unique confirmation code for the completed purchase and associate it with the purchased tickets.

3.2.7.4 Outputs

The system shall display the movie title, theater, showtime, selected seats, purchase total, and confirmation information.

3.2.7.5 Error Handling

If purchase confirmation cannot be generated or displayed, the system shall notify the customer that an error occurred while retrieving the purchase confirmation.

3.2.8 Manage Movies

3.2.8.1 Introduction

The system shall allow authorized administrators to manage movie information.

3.2.8.2 Inputs

The system shall allow authorized administrators to accept, modify, or remove information.

3.2.8.3 Processing

The system shall validate the movie information and update the stored movie data according to an authorized administrator's requested action.

3.2.8.4 Outputs

The system shall display the updated movie information and confirm that the requested change was completed.

3.2.8.5 Error Handling

If the movie information is incomplete or invalid, the system shall notify the administrator and shall not save the changes until the error is corrected.

3.2.9 Manage Showtimes

3.2.9.1 Introduction

The system shall allow authorized administrators to manage movie showtimes.

3.2.9.2 Inputs

The system shall accept showtime information entered, modified, or removed by an authorized administrator.

3.2.9.3 Processing

The system shall validate the movie information and update the stored movie showtime data according to an authorized administrator's requested action.

3.2.9.4 Outputs

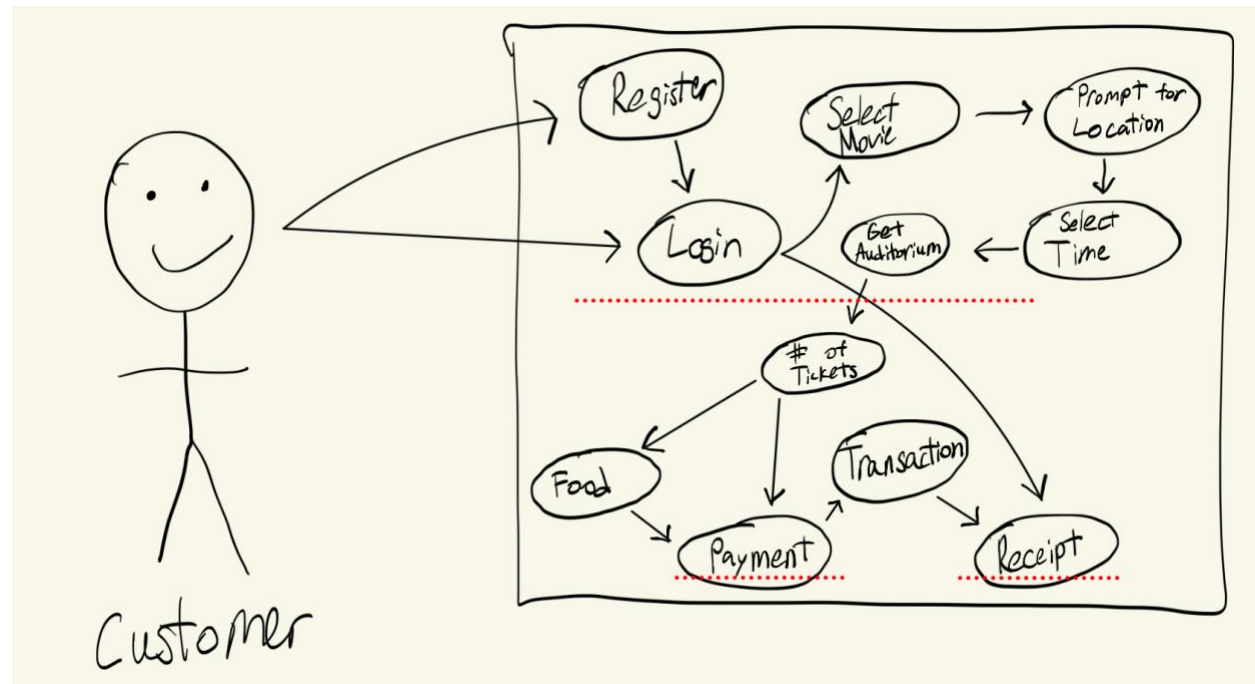
The system shall display the updated movie showtime information and confirm that the requested change was completed.

3.2.9.5 Error Handling

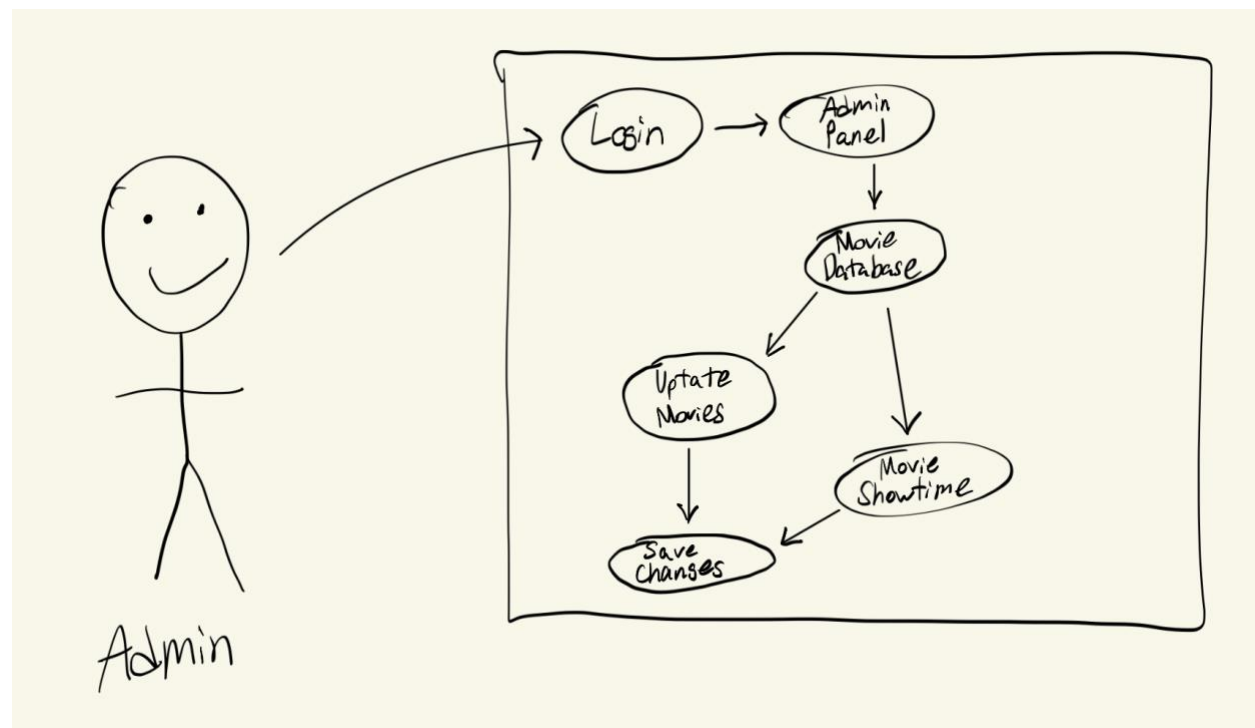
If the movie showtime error is incomplete, invalid, or conflicts with an existing showtime, the system shall notify the administrator and shall not save the changes until the error is corrected.

3.3 Use Cases

3.3.1 Use Case #1



3.3.2 Use Case #2



3.4 Classes / Objects

3.4.1 Login

3.4.1.1 Attributes

- loginID: int
- username: string
- password: string

3.4.1.2 Functions

- validCredentials(): int

3.4.2 Movie

3.4.2.1 Attributes

- movieID: int
- title: string
- duration: int
- ageRating: string
- locationTheater: float
- auditorium: int
- seatNumber: string

3.4.2.2 Functions

- getMovieDetails(): string

3.4.3 Order

3.4.3.1 Attributes

- orderID: int
- selectedMovie: Movie
- totalPrice: float
- food: int

3.4.3.2 Functions

- calculateTotal(): float
- transaction(): int

3.5 Non-Functional Requirements

3.5.1 Performance

- Timely responses to customers actions:
 - Login will be completed in 5 seconds most in normal operating conditions
 - System should display all available movies in 3 seconds most in normal operating conditions
 - System will display all available showtimes form different theatres for the selected movie in 3 seconds most in normal operating conditions
 - System will display the seating information for a selected showtime in 3 seconds most in normal operating conditions
 - System will display options to add snacks and beverages 2 seconds most

- System will display all available snacks and beverages available at selected theatre location n 3 seconds most in normal operating conditions
- System will provide a payment result or indicate the payment is still processing within 10 seconds at most after the customer submits the required payment information in normal operating conditions
- After a successful payment authorization, the system should generate and display the ticket confirmation in 5 seconds at most in normal operating conditions

3.5.2 Reliability

- System will not generate a valid ticket unless payment has been successfully authorized
- The system will not sell the same seat to more than one customer for the same showtime
- The system will preserve confirmed ticket purchases in the event of a system error or temporary interruption

3.5.3 Availability

- System should not remain unavailable for longer than 1 continuous hour
- Planned maintenance will be communicated to users 24 hours before system becomes unavailable
- System gives 20 minutes for selecting and purchasing seats before ending transaction and making seats selectable again

3.5.4 Security

- Sensitive payment information shall be transmitted securely
- System will not display a customer's full card number after payment has been submitted
- System will prevent unauthorized users from accessing another customer's account or ticket information
- User passwords will not be displayed in plain text

3.5.5 Maintainability

- Movie, theatre, showtime, snack and beverage information shall be updateable without requiring changes to the system's source code
- The system's source code will be maintained using a version control system

3.5.6 Portability

- The system will operate on all modern web browsers
- The system will support both desktop and mobile screen sizes
- All core functions and data will remain available and saved across platforms through user login

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2

6. Software Design Specification

6.1 System Description

The Movie Ticketing System is a web based application that allows customers to browse movies, search for nearby theaters, view showtimes, select seats, order food and drinks, and purchase tickets. Customers can also create an account so that their purchases and ticket information can be connected to them. The system supports payment by credit card, debit card, and PayPal. After a purchase is completed, the customer is shown a receipt and can choose to have a copy sent to their email. These features match the scope already defined in the requirements section.

CS250_SRS_copy

The system also includes administrator functions. Administrators are able to manage movie information and showtimes that appear in the application. Theater, movie, showtime, seat, concession, customer, and purchase information will be stored so that it can be retrieved when needed. The system also uses location services to help customers find nearby theaters, with manual search available if location access is not available. CS250_SRS_copy

6.2 Software Architecture Overview

The Movie Ticketing System will be divided into three main parts: the user interface, the application logic, and the database. Separating the system this way makes it easier to organize the code and keep different responsibilities from being mixed together.

The user interface is the part of the system that customers and administrators interact with directly. It will contain the pages or screens for login, movie browsing, theater selection, showtime selection, seat selection, food and drink selection, checkout, and purchase confirmation. Administrators will also have access to interfaces for managing movie and showtime information.

The application logic will handle the actions requested through the user interface. For example, when a customer selects a movie, the application will retrieve the available showtimes. When a customer selects seats, the application will check whether the seats are still available. It will also calculate order totals, process purchases, and create purchase confirmation information.

The database will store the information that needs to remain available between sessions. This includes user accounts, movies, theaters, auditoriums, showtimes, seats, concession items, orders, payments, and tickets. The application logic will communicate with the database whenever information needs to be retrieved or updated.

The system may also communicate with outside services. Location services may be used to determine the customer's current location when searching for nearby theaters. A payment service may be used to authorize customer payments. The existing requirements already include both location based theater searching and external payment processing.

6.2.1 Software Architecture Diagram

The software architecture diagram will show the main components of the Movie Ticketing System and how they communicate with each other.

The diagram should include the following components:

User Interface

- Login and Registration
- Movie Browser
- Theater Browser
- Showtime Selection

Movie Ticketing System

- Seat Selection
- Food and Drink Selection
- Checkout
- Purchase Confirmation
- Administrator Panel

Application Logic

- Authentication
- Movie Management
- Theater Search
- Showtime Management
- Seat Management
- Order Processing
- Payment Processing
- Location Services

Database

- User Data
- Movie Data
- Theater Data
- Showtime Data
- Seat Data
- Concession Data
- Order Data
- Ticket Data

The User Interface will send requests to the Application Logic. The Application Logic will process the request and communicate with the Database when information needs to be retrieved or changed. Location and payment services will connect to the Application Logic instead of directly connecting to the user interface or database.

6.3 UML Class Design

The UML class diagram will describe the main classes used by the Movie Ticketing System. Each class will contain its important attributes and operations. Relationships between classes will also be shown so that it is clear how information moves through the system.

6.3.1 User

The `User` class represents a registered account in the system.

Attributes

Movie Ticketing System

`userID: int`

Stores the unique ID assigned to the user.

`username: string`

Stores the username used to log into the account.

`password: string`

Stores the user's password information.

`email: string`

Stores the email address connected to the account.

Operations

`login(username: string, password: string): bool`

Checks the username and password and returns true if the login is successful.

`logout(): void`

Logs the user out of the system.

6.3.2 Customer

The `Customer` class represents a customer using the system to purchase tickets and concessions. `Customer` will inherit basic account information from the `User` class.

Operations

`createOrder(): Order`

Creates a new order for the customer.

`viewOrders(): List<Order>`

Returns the customer's current and previous orders.

6.3.3 Administrator

The `Administrator` class represents a user with permission to manage movie and showtime information. `Administrator` will also inherit account information from the `User` class.

Operations

`addMovie(movie: Movie): bool`

Adds a new movie to the system.

`updateMovie(movie: Movie): bool`

Updates information for an existing movie.

`removeMovie(movieID: int): bool`

Removes a movie from the system.

Movie Ticketing System

`addShowtime(showtime: Showtime): bool`

Adds a new showtime.

`updateShowtime(showtime: Showtime): bool`

Updates an existing showtime.

`removeShowtime(showtimeID: int): bool`

Removes an existing showtime.

6.3.4 Movie

The `Movie` class stores information about a movie available in the system.

Attributes

`movieID: int`

Stores the unique ID for the movie.

`title: string`

Stores the movie title.

`duration: int`

Stores the movie runtime in minutes.

`ageRating: string`

Stores the age rating of the movie.

`genre: string`

Stores the genre of the movie.

`description: string`

Stores a short description of the movie.

Operations

`getMovieDetails(): string`

Returns the information stored for the movie.

`getShowtimes(theaterID: int): List<Showtime>`

Returns the available showtimes for the movie at a selected theater.

6.3.5 Theater

The `Theater` class represents a participating movie theater.

Attributes

Movie Ticketing System

`theaterID: int`

Stores the unique ID for the theater.

`name: string`

Stores the theater name.

`address: string`

Stores the theater address.

`latitude: double`

Stores the latitude of the theater.

`longitude: double`

Stores the longitude of the theater.

Operations

`getMovies(): List<Movie>`

Returns the movies currently available at the theater.

`getShowtimes(movieID: int): List<Showtime>`

Returns showtimes for a selected movie.

`getConcessions(): List<ConcessionItem>`

Returns the food and drink items available at the theater.

6.3.6 Auditorium

The `Auditorium` class represents a screening room inside a theater.

Attributes

`auditoriumID: int`

Stores the unique ID for the auditorium.

`theaterID: int`

Stores the ID of the theater that contains the auditorium.

`auditoriumNumber: int`

Stores the number assigned to the auditorium.

`capacity: int`

Stores the number of seats in the auditorium.

Operations

`getSeats(): List<Seat>`

Returns the seats assigned to the auditorium.

6.3.7 Showtime

The `Showtime` class represents a scheduled showing of a movie.

Attributes

`showtimeID: int`

Stores the unique ID for the showtime.

`movieID: int`

Stores the ID of the movie being shown.

`auditoriumID: int`

Stores the ID of the auditorium where the movie will be shown.

`startTime: DateTime`

Stores the date and starting time of the showing.

Operations

`getAvailableSeats(): List<Seat>`

Returns the seats that are currently available for the showtime.

`isValid(): bool`

Checks whether the showtime is valid.

6.3.8 Seat

The `Seat` class represents a seat inside an auditorium.

Attributes

`seatID: int`

Stores the unique ID for the seat.

`auditoriumID: int`

Stores the auditorium that contains the seat.

`row: string`

Stores the row of the seat.

`seatNumber: int`

Stores the number of the seat.

Operations

`isAvailable(showtimeID: int): bool`

Checks whether the seat is available for a selected showtime.

6.3.9 ConcessionItem

The `ConcessionItem` class represents a food or drink item available at a theater. Food and drink ordering is already included in the current requirements. CS250_SRS_copy

Attributes

`itemID: int`

Stores the unique ID for the item.

`theaterID: int`

Stores the theater that sells the item.

`name: string`

Stores the item name.

`price: float`

Stores the price of the item.

`available: bool`

Stores whether the item is currently available.

Operations

`isAvailable(): bool`

Checks whether the item is available for purchase.

`getPrice(): float`

Returns the price of the item.

6.3.10 Order

The `Order` class represents everything the customer is purchasing in one transaction.

Attributes

`orderID: int`

Stores the unique ID for the order.

`customerID: int`

Stores the ID of the customer who created the order.

`tickets: List<Ticket>`

Stores the tickets included in the order.

`concessions: List<ConcessionItem>`

Stores the concession items included in the order.

Movie Ticketing System

`totalPrice: float`

Stores the total price of the order.

`status: string`

Stores the current status of the order.

Operations

`addTicket(ticket: Ticket): void`

Adds a ticket to the order.

`addConcession(item: ConcessionItem): void`

Adds a concession item to the order.

`removeTicket(ticketID: int): void`

Removes a ticket from the order.

`removeConcession(itemID: int): void`

Removes a concession item from the order.

`calculateTotal(): float`

Calculates and returns the total price of the order.

`completeOrder(): bool`

Attempts to complete the order and returns whether it was successful.

6.3.11 Ticket

The `Ticket` class represents a purchased seat for a specific showtime.

Attributes

`ticketID: int`

Stores the unique ID for the ticket.

`showtimeID: int`

Stores the showtime connected to the ticket.

`seatID: int`

Stores the seat connected to the ticket.

`price: float`

Stores the price of the ticket.

`status: string`

Stores the current status of the ticket.

Operations

Movie Ticketing System

`generateTicket(): string`

Returns the ticket information after a successful purchase.

6.3.12 Payment

The `Payment` class represents the payment made for an order.

Attributes

`paymentID: int`

Stores the unique ID for the payment.

`orderId: int`

Stores the order connected to the payment.

`amount: float`

Stores the payment amount.

`paymentMethod: string`

Stores the payment method used, such as credit card, debit card, or PayPal.

`status: string`

Stores the current payment status.

Operations

`processPayment(): bool`

Attempts to process the payment and returns whether it was successful.

`refund(amount: float): bool`

Attempts to refund the specified amount and returns whether the refund was successful.

6.3.13 Class Relationships

The `Customer` and `Administrator` classes inherit from the `User` class. A `Theater` can contain multiple `Auditorium` objects and can offer multiple `ConcessionItem` objects. A `Movie` can have multiple `Showtime` objects, while each showtime belongs to one movie and one auditorium. An auditorium can contain multiple seats.

A customer can create multiple orders. Each order can contain multiple tickets and concession items. Each order is connected to a payment. A ticket is connected to one showtime and one seat. These relationships allow the system to keep track of what movie the customer is seeing, where the movie is being shown, what seat was purchased, and what other items were included in the order.

6.4 Development Plan and Timeline

The development of the Movie Ticketing System will be divided among the team members based on the main parts of the project. The first part of development will focus on finalizing the design, database structure, and class relationships. After the basic structure is complete, the team will begin implementing the customer and administrator features.

Movie and theater browsing, showtimes, seating, concessions, account management, and checkout will be developed as separate parts of the application. These parts will then be connected and tested together. The final stage of development will focus on fixing problems, improving the interface, and completing the project documentation.

6.4.1 Task Partitioning

Task	Team Member	Estimated Time
Review requirements and design	All members	Week 1
Software architecture diagram	Cameron Penaranda	Week 1
UML class diagram	Justin Paul Liporada	Week 1
Database design	Ayaan Bhatkar	Week 1
Login and account system	Cameron Penaranda	Week 2
Movie and theater browsing	Justin Paul Liporada	Week 2
Showtime and seat selection	Ayaan Bhatkar	Week 2 to 3
Food and drink ordering	Justin Paul Liporada	Week 3
Checkout and payment	Cameron Penaranda	Week 3 to 4
Database integration	Ayaan Bhatkar	Week 3 to 4
System testing	All members	Week 5
Bug fixes and final changes	All members	Week 6
Final documentation	All members	Week 6

6.4.2 Team Member Responsibilities

Cameron Penaranda will mainly work on the software architecture, account system, and checkout and payment functionality. Justin Paul Liporada will mainly work on the UML design, movie and theater browsing, and food and drink functionality. Ayaan Bhatkar will mainly work on the database, showtime functionality, seat selection, and database integration.

All team members will help with testing, reviewing changes, fixing bugs, and updating the project documentation. Each team member will also make commits to the project GitHub repository throughout development.